SYSTEM ANALYSIS DOCUMENTATION (SAD)
UMHackathon 2026

TEAM NAME: HairLoss

GROUP MEMBERS:

TEOH WEI CHENG

WOON YAO MING

GAN YI MING

Table of Content

Project: Auto-Wargame (Automated Business & Policy Wargaming Orchestrator)

1. Introduction

This section introduces the strategic solution for the identified problem statement highlighted in the Product Requirement Document.

1.1. Purpose

The system analysis document highlights the technical scope and design-related decisions behind the Auto-Wargame development. The key elements of the system covered are as follows:

- **Architecture:**
    a. It is a web-based, cloud-ready enterprise platform. The core services are managed by an Agentic Workflow state machine acting independently.
- **Data Flows:**
    a. Unstructured "What-If" scenario inputs flow from the web interface, move through the backend API Orchestrator, dynamically fetch mock macro-data, and return a structured strategic impact report.
- **Model Process:**
    a. The model shows the end-to-end workflow of the core features: intent parsing, multi-step API reasoning, handling data failures via proxy inference, and final dashboard generation.
- **Role of Reference:**
    a. This document is a reference for all the developers and testers to enable the overall high-level architecture of this project.

1.2. Background

Across Malaysia, corporate strategy teams and government policymakers rely on fragmented, manual, and unstructured workflows to model the impact of future macroeconomic events.

- **1. Previous Version:** Existing workflows rely heavily on manual human interpretation. When specific data is missing or ambiguous, traditional automated pipelines crash, stall, or fail completely.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Policy Analysts / C-Suite	Input unstructured hypothetical scenarios and review the final strategic impact reports.	A fast, autonomous "strategic brain" that replaces days of manual research with a structured dashboard generated in minutes.
Development Team	Build the platform, maintain the state machine, and manage the API tool layer.	Clear API usage contracts, a modular codebase, and a well-documented prompt engineering architecture.
QA Team	Validate the system behavior, specifically focusing on workflow fault tolerance.	Defined coverage of edge cases, specifically verifying the system's "Graceful Degradation" when Mock APIs return 404 errors.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. **Overview**

Type	**Details**
System	Web-based Enterprise Application (Streamlit/Gradio)
Architecture	Client-Server structure interacting with external AI Services and Mock Micro-APIs

Auto-Wargame is structured as a client-server-based system. The web interface communicates with a Python backend layer, which acts as the stateful workflow engine. This backend manages interactions with the Z.AI GLM and external macro-data APIs.

2.1.2. **LLM as Service Layer**

The architecture integrates Z.AI's GLM not as a standard chatbot, but as the central reasoning engine within the service layer.

2.1.3. **Dependency Diagram**

a. User intents are parsed, wrapped into strict System Prompts demanding JSON output, and sent to the GLM API.
b. The context window contains the original unstructured query, the current step of the "Investigation Plan", and historical records of successful or failed API calls.
c. The system receives GLM's function-calling responses, parses the required parameters, and passes them to the internal API Orchestrator to fetch data.
d. Token limitations are enforced by summarizing previous investigation steps before injecting them back into the context window.
e. Major API calls include the Frontend requesting workflow updates from the Backend, and the Backend routing queries to either the GLM or external Mock APIs.
f. The Backend logic strictly governs the state machine, ensuring the GLM cannot output a final report until the required data points are successfully fetched from the external databases.

2.2. Technological Stack

2.2.1. Frontend: Streamlit (Chosen for rapid UI prototyping to focus development time on workflow logic).

2.2.2. Backend: Python / FastAPI (To handle state management and RESTful routing).

2.2.3. Database: In-memory state tracking and lightweight JSON/SQLite databases for Mock APIs.

2.2.4. Cloud/Deployment: Local environment or lightweight cloud container for MVP demonstration.

2.3. Key Data Flows

2.3.1. **Data Flow Diagram (DFD):**

Illustrates data movement from unstructured user prompt -> Intent Parser -> API Orchestrator -> Adaptive Edge-Case Engine (if initial API fails) -> Final Dashboard Generation.

2.3.2. **Normalized Database Schema:**

The Mock APIs utilize relational mapping between years, historical events, and economic indicators to ensure the GLM receives logically sound data during retrieval.

3. Functional Requirements & Scope

This section highlights the core boundaries of the project, enabling the team to focus on a high-impact MVP to showcase the technical feasibility of the Agentic Workflow.

3.1. Minimum Viable Product (MVP)

#	Features	Description
1	GLM Intent Parser	Ingests unstructured text and converts it into a structured "Investigation Plan" consisting of sub-tasks.
2	Dynamic API Orchestrator	Executes function-calling capabilities, allowing the GLM to query specific mock databases (e.g., inflation rates).
3	Adaptive Edge-Case Engine	The core fallback logic. If an API returns a 404 error, it triggers the GLM to search for historical proxy metrics.
4	"Chain of Thought" UI	A live terminal side-panel that visually prints the GLM's internal multi-step reasoning processes.
5	Strategic Dashboard	Outputs the final multi-variable strategic impact report in a structured Markdown/JSON format.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	The workflow must successfully manage data ambiguity. It cannot crash if primary data is missing.	Trigger the "Graceful Degradation / Proxy Inference" state in 100% of cases where API responses fail.
Token Latency	The multi-step reasoning loop must be completed within 60 seconds.	Limit the maximum depth of the reasoning chain (e.g., 5-step loop limit) to prevent infinite loops.
Cost Efficiency	Prevent token exhaustion due to model hallucination loops.	Enforce strict system prompts limiting GLM output and utilize API error caching.

3.3. Out of Scope / Future Enhancements

a. Direct execution of real-world actions (e.g., automatically executing trades or modifying supply chains based on the report).
b. Integration with expensive, live financial terminals like Bloomberg (MVP relies on controlled Mock APIs).
c. B2C consumer mobile app deployment.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. **Testing Strategy:**

We will perform deliberate "Fault Injection" testing, intentionally disabling Mock APIs to prove the workflow smoothly transitions into proxy-data retrieval mode without breaking.

4.1.2. **Priority Matrix**:

P1 Critical: If the GLM attempts to "hallucinate" an answer without citing a specific Mock API response, the system must forcefully interrupt the workflow and prompt a retry.

4.2. Monitoring

Track the exact number of successful API tool calls versus fallback loops initiated during a user session to demonstrate workflow efficiency.

4.3. Assumptions

a. The Z.AI GLM API will have sufficiently low latency and stable availability to complete reasoning loops within the strict demonstration timeframes.
b. Judges will accept the use of hardcoded Mock APIs as a valid demonstration of the dynamic workflow logic.

4.4 External Dependencies

Tools	**Purpose**	**Risks**
Z.AI GLM API	The core LLM reasoning engine handles multi-step logic and structured output generation.	High - Rate limitations could cause workflow failure. Must implement error handling and retry logic.
Macroeconomic Mock APIs	Simulates real-world databases (DOSM, BNM) to supply variables for the sandbox environment.	Low - Hardcoded local endpoints ensure high availability during the live hackathon pitch.

5. Project Management & Team Contributions

5.1. Project Timeline

- **Days 1-2:** System architecture planning and mock database construction.
- **Days 3-4:** Development of the core stateful workflow engine and GLM prompt engineering.
- **Days 5-6:** Streamlit UI integration and "Chain of Thought" terminal logging.
- **Day 7:** Intensive edge-case testing and live pitch rehearsal.

5.2. Team Members Role

- **Gan Yi Ming:** System Architecture & Backend Logic – Designing the state machine utilizing Object-Oriented Programming (OOP) principles to ensure clear relationships between workflow stages.
- **Woon Yao Ming:** Prompt Engineering & API Orchestration – Tuning the GLM system instructions to strictly adhere to API tool-calling formats and avoid hallucinations.
- **Teoh Wei Cheng:** Frontend & Dashboard Design – Constructing the Streamlit interface to visualize the macro-level impact reports effectively.

5.3. Recommendations

During the live Hackathon presentation, the team is recommended to utilize highly specific, pre-planned "What-If" scenarios (such as the RON95 subsidy cancellation). This guarantees a flawless demonstration of the backend routing logic, data cross-referencing, and adaptive error handling.